# LocalActionDiagrams

## Super awesome package for local action diagrams.

0.1

29 January 2023

**Marcus Chijoff**
**Stephan Tornier**

**Marcus Chijoff** Email: marcus.chijoff@uon.edu.au
Homepage: https://newcastle.edu.au
Address: No

**Stephan Tornier** Email: stephan.tornier@newcastle.edu.au
Homepage: https://newcastle.edu.au
Address: No

# Contents

# Chapter 1

# RSGraphs

An RSGraph is a graph $\Gamma = (V, A, o, r)$ that consists of a vertex set $V$, an arc set $A$, a map $o : A \to V$ which assigns each vertex to an *origin*, and a bijection $r : A \to A$ which assigns each arc to a *reverse* arc. There is also a map $t := o \circ r$ which assigns each arc to a *terminus* vertex. Note the RSGraphs are allowed to contain loops and parallel arcs.

We created the RSGraph object for this package due to our use of graphs having a specialised definition. For example, every arc in an RSGraph must have a reverse arc associated with it while a *Digraph* from the Digraphs allows for arcs that do not have an associated reverse arc. Creating a new object category for RSGraphs allows for both better error checking when using RSGraphs and to avoid conflicting definitions, such as the definition of a cycle for a Digraph and an RSGraph. We provide functionality to convert between an RSGraph and a *Graph* from the GRAPE package or a *Digraph* from the Digraphs package.

The name "RSGraph" arises from Reid and Smith, the authors of the local action diagram paper. This name was chosen to avoid conflicting with the GRAPE package.

## 1.1 Creating RSGraphs

### 1.1.1 IsRSGraph (for an object)

▷ `IsRSGraph(obj)` (Category)

**Returns:** `true` if *graph* is of the category `IsRSGraph` and `false` otherwise.

Every RSGraph object belongs to the category `IsRSGraph`. Furthermore, every RSGraph is an immutable attribute storing object.

### 1.1.2 RSGraphByAdjacencyList (for a list, permutation[, and list])

▷ `RSGraphByAdjacencyList(adjacency_list, reverse_map[, vertex_ids])` (operation)

▷ `RSGraphByAdjacencyListNC(adjacency_list, reverse_map[, vertex_ids])` (operation)

**Returns:** An RSGraph.

This function creates an RSGraph described by an adjacency list. The argument *adjacnecy_list* is a list of the from `[[origin, terminus], ...]`. Each arc in the graph is given an id which corresponds to its position in the list *adjacency_list*.

The reverse map must be a permutation that is an involution --- i.e. `reverse_map*reverse_map = ()`. Furthermore, if element *idx* of *adjacency_list* is `[u, v]` then *adjacency_list[idx^reverse_map]* must equal `[v, u]`.

By default, the vertices are labelled with ids $\{1,2,\ldots,n\}$ where $n$ is the maximal element of `Flat(adjacency_list)`. If the optional third argument *`vertex_ids`* is given then these are taken to be the vertex ids. This argument must be a dense list of integers.

Finally, the underlying graph must be connected.

The NC variant of the function does not check that the graph is connected, the reverse map is valid, and that each arc has an associated reverse arc.

Example

```
gap> adj_list := [[1, 2], [2, 1], [2, 2], [2, 2], [1, 1]];;
gap> rev_map := (1,2)(3,4);;
gap> graph := RSGraphByAdjacencyList(adj_list, rev_map);
<RSGraph with 2 vertices and 5 arcs>
gap> adj_list2 := [[2, 2]];;
gap> rev_map2 := ();;
gap> vertex_ids := [2];;
gap> graph := RSGraphByAdjacencyList(adj_list2, rev_map2, vertex_ids);
<RSGraph with 1 vertex and 1 arc>
```

### 1.1.3 RSGraphByAdjacencyMatrix (for a matrix, permutation[, and list])

▷ `RSGraphByAdjacencyMatrix(`*`adjacency_matrix, reverse_map[, vertex_ids]`*`)` (operation)
▷ `RSGraphByAdjacencyMatrixNC(`*`adjacency_matrix, reverse_map[, vertex_ids]`*`)` (operation)

**Returns:** An RSGraph.

This function creates an RSGraph described by an adjacency matrix. The argument *`adjacnecy_matrix`* is an $n \times n$ matrix represented by a list of lists. This means that there are $m$ arcs between vertices $u$ and $v$ of the graph if `adjacency_matrix[u][v] = m`.

The matrix is read in row-major order to determine the arc ids of the graph. This is important for determining the reverse map of the graph. The reverse map must be a permutation that is an involution --- i.e. `reverse_map*reverse_map = ()`.

By default, the vertices are labelled with ids $\{1,2,\ldots,n\}$ where $n$ `Size(adjacency_matrix)` --- i.e. the number of rows (and columns) of the matrix. If the optional third argument *`vertex_ids`* is given then these are taken to be the vertex ids. This argument must be a dense list of integers and element `idx` of the list will be the label of the vertex in row and column `idx` of *`adjacency_matrix`*.

Finally, the underlying graph must be connected.

Note that the first argument must be a list of lists with each sublist containing integers. In particular, it can not be an object in the category `IsMatrixObj` constructed by the function `Matrix`.

The NC variant of the function does not check that the graph is connected, the reverse map is valid, and that each arc has an associated reverse arc.

Example

```
gap> adj_mat := [[1, 2], [2, 0]];;
gap> rev_map := (2, 4)(3,5);;
gap> vertex_ids := [2, 3];;
gap> graph := RSGraphByAdjacencyMatrix(adj_mat, rev_map, vertex_ids);
<RSGraph with 2 vertices and 5 arcs>
gap> Print(graph);
Vertices = { 2, 3 }
Arcs = {
        1 = ( origin = 2, terminus = 2, inverse = 1 )
        2 = ( origin = 2, terminus = 3, inverse = 4 )
```

```
        3 = ( origin = 2, terminus = 3, inverse = 5 )
        4 = ( origin = 3, terminus = 2, inverse = 2 )
        5 = ( origin = 3, terminus = 2, inverse = 3 )
}
Reverse Map = (2,4)(3,5)
```

## 1.2 RSGraph Attributes and Properties

### 1.2.1 RSGraphVertices

▷ RSGraphVertices(*graph*) (attribute)

**Returns:** The list of vertex ids of *graph*.

### 1.2.2 RSGraphNumberVertices

▷ RSGraphNumberVertices(*graph*) (attribute)

**Returns:** The number of vertices of *graph*.

### 1.2.3 RSGraphArcs

▷ RSGraphArcs(*graph*) (attribute)

**Returns:** The record of arcs of *graph*.

The name of each component of the record is the id of the arc. Each component it itself a record with three components: origin, terminus, and inverse. These store the origin vertex, terminus vertex, and inverse arc id respectively.

For iterating over the arcs of an RSGraph see RSGraphArcIterator (1.3.1).

Example

```
gap> adj_list := [[1, 2], [2, 1], [2, 2], [2, 2], [1, 1]];;
gap> rev_map := (1,2)(3,4);;
gap> graph := RSGraphByAdjacencyList(adj_list, rev_map);
<RSGraph with 2 vertices and 5 arcs>
gap> RSGraphArcs(graph).1;
rec( inverse := 2, origin := 1, terminus := 2 )
```

### 1.2.4 RSGraphNumberArcs

▷ RSGraphNumberArcs(*graph*) (attribute)

**Returns:** The number of arcs of *graph*.

### 1.2.5 RSGraphArcIDs

▷ RSGraphArcIDs(*graph*) (attribute)

**Returns:** The list of arc ids of *graph*.

### 1.2.6 RSGraphReverseMap

▷ RSGraphReverseMap(*graph*) (attribute)

**Returns:** The reverse map of *graph*.

### 1.2.7 RSGraphAdjacencyMatrix

▷ `RSGraphAdjacencyMatrix(`*graph*`)` (attribute)

**Returns:** The adjacency matrix of *graph*.

### 1.2.8 RSGraphOutNeighbours

▷ `RSGraphOutNeighbours(`*graph*`)` (attribute)

**Returns:** The record of vertex neighbours of *graph*.

The name of each component is a vertex id of the graph. Each element is a list of vertex ids corresponding to the neighbours of the vertex. A vertex id `v_id` is in list `idx` if and only if there is an arc connecting the vertices `idx` and `v_id`. For example, if there is an arc from vertex 1 to 3 of the graph then 3 is an element of `RSGraphOutNeighbours(graph).1`.

### 1.2.9 RSGraphInNeighbours

▷ `RSGraphInNeighbours(`*graph*`)` (attribute)

**Returns:** The record of vertex neighbours of *graph*.

Since every arc has an inverse arc this returns the same result as `RSGraphOutNeighbours(graph)`.

### 1.2.10 RSGraphOutArcs

▷ `RSGraphOutArcs(`*graph*`)` (attribute)

**Returns:** The record of arcs originating at each vertex.

The name of each component is a vertex id of the graph. Each element is a list of arc ids. An arc id is in list `idx` if and only if the origin of that arc is `idx`. For example, if an arc with id 2 originates at vertex 3 then 2 is an element of `RSGraphInNeighbours(graph).3`.

### 1.2.11 RSGraphInArcs

▷ `RSGraphInArcs(`*graph*`)` (attribute)

**Returns:** The record of arcs originating at each vertex.

The name of each component is a vertex id of the graph. Each element is a list of arc ids. An arc id is in list `idx` if and only if the terminus of that arc is `idx`. For example, if an arc with id 2 terminates at vertex 1 then 2 is an element of `RSGraphInNeighbours(graph).1`.

Example

```
gap> adj_list := [[1, 2], [2, 1], [2, 2], [2, 2]];;
gap> rev_map := (1,2)(3,4);;
gap> graph := RSGraphByAdjacencyList(adj_list, rev_map);
<RSGraph with 2 vertices and 4 arcs>
gap> RSGraphOutNeighbours(graph);
rec( 1 := [ 2 ], 2 := [ 1, 2 ] )
gap> RSGraphInNeighbours(graph);
rec( 1 := [ 2 ], 2 := [ 1, 2 ] )
gap> RSGraphOutArcs(graph);
rec( 1 := [ 1 ], 2 := [ 2, 3, 4 ] )
gap> RSGraphInArcs(graph);
rec( 1 := [ 2 ], 2 := [ 1, 3, 4 ] )
```

### 1.2.12 RSGraphBipartition

▷ `RSGraphBipartition(`*graph*`)` (attribute)

**Returns:** The bipartition of `graph` if it is bipartite and `fail` otherwise.

If the graph is bipartite then this returns a list containing two lists. These two lists contain the vertex ids in each bipartition.

Note that for an RSGraph to be bipartite it must not contain parallel edges or loops.

Example

```
gap> adj_list := [[1, 2], [2, 1], [1, 3], [3, 1]];;
gap> rev_map := (1,2)(3,4);;
gap> graph := RSGraphByAdjacencyList(adj_list, rev_map);
<RSGraph with 3 vertices and 4 arcs>
gap> RSGraphBipartition(graph);
[ [ 1 ], [ 2, 3 ] ]
gap> adj_list := [[1, 2], [2, 1], [2, 2], [2, 2]];;
gap> rev_map := (1,2)(3,4);;
gap> graph := RSGraphByAdjacencyList(adj_list, rev_map);
<RSGraph with 2 vertices and 4 arcs>
gap> RSGraphBipartition(graph);
fail
```

### 1.2.13 RSGraphDegree

▷ `RSGraphDegree(`*graph*`)` (attribute)

**Returns:** The maximum number of arcs originating/terminating at any vertex in the graph.

### 1.2.14 RSGraphIsCycle

▷ `RSGraphIsCycle(`*graph*`)` (property)

**Returns:** `true` if *graph* is a cycle graph and `false` otherwise.

This is a cycle in the sense of !!!cite RS paper here!!!. A graph is a cycle if:

- It has a single vertex, two arcs, and a non-trivial reverse map.
- It has two vertices and two edges (i.e. four arcs) between them.
- It has three or more vertices, no loops, and all vertices have degree two.

### 1.2.15 RSGraphIsBipartite

▷ `RSGraphIsBipartite(`*graph*`)` (property)

**Returns:** `true` if *graph* is a bipartite graph and `false` otherwise.

This is equivalent to `RSGraphBipartition(graph) <> fail`.

### 1.2.16 RSGraphHasParallelArcs

▷ `RSGraphHasParallelArcs(`*graph*`)` (property)

**Returns:** `true` if *graph* has multiple arcs in the same direction between any two pairs of vertices.

## 1.3 RSGraph Operations

### 1.3.1 RSGraphArcIterator

▷ RSGraphArcIterator(*graph*) (operation)

**Returns:** An iterator which iterates over the arcs of *graph*.

This functions returns an iterator object which iterates over the arcs of *graph*. Each element of the iterator is of the form [arc_id, arc_record] where arc_record is in the form described in RSGraphArcs (1.2.3).

Example

```
gap> adj_list := [[1, 2], [2, 1], [2, 2]];;
gap> rev_map := (1,2);;
gap> graph := RSGraphByAdjacencyList(adj_list, rev_map);;
gap> for arc in RSGraphArcIterator(graph) do
>     View(arc);
>     Print("\n");
> od;
[ 1, rec( inverse := 2, origin := 1, terminus := 2 ) ]
[ 2, rec( inverse := 1, origin := 2, terminus := 1 ) ]
[ 3, rec( inverse := 3, origin := 2, terminus := 2 ) ]
```

### 1.3.2 RSGraphSubgraph (for an RSGraph and List of arcs)

▷ RSGraphSubgraph(*graph, arc_list*) (operation)

▷ RSGraphSubgraphNC(*graph, arc_list*) (operation)

**Returns:** An arc induced subgraph of *graph*.

Given a graph and list of arcs in this graph, this function returns the subgraph consisting exactly of those arcs and the vertices connecting them. The vertex ids, arc ids, and reverse map are preserved. Note that the reverse map is not changed and so may act on a larger set of integers than is strictly necessary.

If an arc is in *arc_list* then its reverse must also be in it. Furthermore, the resulting subgraph must be connected.

The NC variant of this function does not check for connectivity or for the inclusion of arc reversals.

Example

```
gap> adj_list := [[1, 2], [2, 1], [1, 2], [2, 1], [2, 2]];;
gap> rev_map := (1,2)(3,4);;
gap> graph := RSGraphByAdjacencyList(adj_list, rev_map);;
gap> Print(graph);
Vertices = { 1, 2 }
Arcs = {
        1 = ( origin = 1, terminus = 2, inverse = 2 )
        2 = ( origin = 2, terminus = 1, inverse = 1 )
        3 = ( origin = 1, terminus = 2, inverse = 4 )
        4 = ( origin = 2, terminus = 1, inverse = 3 )
        5 = ( origin = 2, terminus = 2, inverse = 5 )
}
Reverse Map = (1,2)(3,4)
gap> subgraph := RSGraphSubgraph(graph, [1,2,5]);
<RSGraph with 2 vertices and 3 arcs>
```

```
gap> Print(subgraph);
Vertices = { 1, 2 }
Arcs = {
        1 = ( origin = 1, terminus = 2, inverse = 2 )
        2 = ( origin = 2, terminus = 1, inverse = 1 )
        5 = ( origin = 2, terminus = 2, inverse = 5 )
}
Reverse Map = (1,2)(3,4)
```

### 1.3.3 RSGraphToStandardForm

▷ RSGraphToStandardForm(*graph*) (operation)

**Returns:** A record containing the graph in a standard range and the maps from the old graph to the new graph.

Given an arbitrary RSGraph with $N$ vertices and $M$ arcs this function changes the vertex ids to be in $\{1,2,\ldots,N\}$ and arc ids to be in the range $\{1,2,\ldots,M\}$. It also ensures that the arcs are sorted in lexicographical order with respect to the origin and terminus vertices.

If $\Gamma$ is the *graph* then the vertex mapping is defined by $V(\Gamma)_i \mapsto i$. The arcs origin and terminus vertices then have this mapping applied to them. The resulting arcs are then sorted in lexicographical order so that an arc with a smaller arc id will have an origin vertex with a smaller id than one with a larger arc id. If the origin vertices are equal then the arc with the smaller terminus id will have a smaller arc id. The reverse map of the graph is changed to have the same action on the new arc ids.

This function returns a record containing three components. The `graph` component contains the new graph constructed from it, the `vertex_id_map` component contains the map from the vertex ids of the original graph to the new graph, and the `arc_id_map` component contains the map from the arc ids of the original graph to the new graph.

The `vertex_id_map` and `arc_id_map` are both in the category `IsConstantTimeAccessGeneralMapping`.

Example

```
gap> adj_list := [[2, 3], [3, 2], [2, 3], [3, 2], [3, 3]];;
gap> rev_map := (1,2)(3,4);;
gap> vertex_ids := [2, 3];;
gap> graph := RSGraphByAdjacencyList(adj_list, rev_map, vertex_ids);;
gap> Print(graph);
Vertices = { 2, 3 }
Arcs = {
        1 = ( origin = 2, terminus = 3, inverse = 2 )
        2 = ( origin = 3, terminus = 2, inverse = 1 )
        3 = ( origin = 2, terminus = 3, inverse = 4 )
        4 = ( origin = 3, terminus = 2, inverse = 3 )
        5 = ( origin = 3, terminus = 3, inverse = 5 )
}
Reverse Map = (1,2)(3,4)
gap> standard_rec := RSGraphToStandardForm(graph);;
gap> Print(standard_rec.graph);
Vertices = { 1, 2 }
Arcs = {
        1 = ( origin = 1, terminus = 2, inverse = 3 )
        2 = ( origin = 1, terminus = 2, inverse = 4 )
        3 = ( origin = 2, terminus = 1, inverse = 1 )
```

```
        4 = ( origin = 2, terminus = 1, inverse = 2 )
        5 = ( origin = 2, terminus = 2, inverse = 5 )
}
Reverse Map = (1,3)(2,4)
gap> Print(2^standard_rec.arc_id_map = 3);
3
```

### 1.3.4 RSGraphSpanningTree

▷ `RSGraphSpanningTree(`*`graph[, search_type]`*`)` (operation)

**Returns:** A spanning tree of the graph.

This function calculates a spanning tree of the graph based on the edges. This means it does not include and loops or cycles (see `RSGraphIsCycle` (1.2.14) for the definition of a cycle in an RSGraph). By default it uses a breadth first search.

If the optional parameter *`search_type`* is equal to the string `"dfs"` then it will use a depth first search instead. If it is equal to the string `"bfs"` then it will use a breadth first search. If it is equal to any other string then a warning will be raised and the function will use a breadth first search.

Example

```
gap> adj_list := [[1, 2], [2, 1], [1, 3], [3, 1], [1, 4], [4, 1], \
>                 [2, 3], [3, 2], [3, 4], [4, 3]];;
gap> rev_map := (1,2)(3,4)(5,6)(7,8)(9,10);;
gap> graph := RSGraphByAdjacencyList(adj_list, rev_map);;
gap> tree_bfs := RSGraphSpanningTree(graph);;
gap> Print(tree_bfs);
Vertices = { 1, 2, 3, 4 }
Arcs = {
        1 = ( origin = 1, terminus = 2, inverse = 2 )
        2 = ( origin = 2, terminus = 1, inverse = 1 )
        3 = ( origin = 1, terminus = 3, inverse = 4 )
        4 = ( origin = 3, terminus = 1, inverse = 3 )
        5 = ( origin = 1, terminus = 4, inverse = 6 )
        6 = ( origin = 4, terminus = 1, inverse = 5 )
}
Reverse Map = (1,2)(3,4)(5,6)(7,8)(9,10)
gap> tree_dfs := RSGraphSpanningTree(graph, "dfs");;
gap> Print(tree_dfs);
Vertices = { 1, 2, 3, 4 }
Arcs = {
        1 = ( origin = 1, terminus = 2, inverse = 2 )
        2 = ( origin = 2, terminus = 1, inverse = 1 )
        7 = ( origin = 2, terminus = 3, inverse = 8 )
        8 = ( origin = 3, terminus = 2, inverse = 7 )
        9 = ( origin = 3, terminus = 4, inverse = 10 )
        10 = ( origin = 4, terminus = 3, inverse = 9 )
}
Reverse Map = (1,2)(3,4)(5,6)(7,8)(9,10)
```

# Chapter 2

# Local Action Diagrams

A local action diagram $\Delta = (\Gamma, (X_a), (G(v)))$ consists of:

- a graph $\Gamma$ (an `RSGraph`),
- for each arc $a \in A(\Gamma)$a *colour set* $|X_a|$ disjoint from every other colour set,
- for each vertex $v \in V(\Gamma)$ a group $G(v)$ labelling the vertex such that $G(V)$ acts on $\bigsqcup_{a \in o^{-1}(v)} X_a$ and the orbits of $G(V)$ are precisely the colour sets $|X_a|$ for $a \in o^{-1}(v)$.

Local action diagrams are a powerful tool for studying $(P)$-closed groups because isomorphism classes of local action diagrams are in a one-to-one correspondence with conjugacy classes of $(P)$-closed groups. Furthermore, properties of the corresponding $(P)$-closed group can be determined from the local action diagram. In the case when a local action diagram is finite, we can represent the diagram in GAP and determine properties of the (potentially infinite) corresponding group.

## 2.1 Creating Local Action Diagrams

### 2.1.1 IsLocalActionDiagram (for an object)

▷ `IsLocalActionDiagram(`*obj*`)` (Category)

**Returns:** `true` if the object is of the category `IsLocalActionDiagram` and false otherwise.

Every local action diagram belongs to the category `IsLocalActionDiagram`. Furthermore, every local action diagram is an immutable attribute storing object.

### 2.1.2 PermGroupDomain (for a Permutation Group)

▷ `PermGroupDomain(`*perm_group*`)` (attribute)

**Returns:** The domain the permutation group acts on.

By default, a permutation group *perm_group* in GAP is treated as acting on `MovedPoints(`*perm_group*`)`. Often when working with local action diagrams we need to consider groups with an action that has fixed points. To do this we provided the `PermGroupDomain` attribute form permutation groups.

This is a mutable attribute that is treated as the set the permutation group acts on for all functions implemented in this package. By default it is equal to `MovedPoints(`*perm_group*`)`.

Note that this attribute does not affect any functions not implemented in this package. For example, the default acting set for the `Orbits` function is still `MovedPoints(`*perm_group*`)`.

Example

```
gap> group := Group((2, 3));;
gap> Orbits(group);
[ [ 2, 3 ] ]
gap> PermGroupDomain(group);
[ 2, 3 ]
gap> SetPermGroupDomain(group, [1, 2, 3]);;
gap> Orbits(group);
[ [ 2, 3 ] ]
gap> Orbits(group, PermGroupDomain(group));
[ [ 1 ], [ 2, 3 ] ]
```

### 2.1.3 LocalActionDiagramFromData (for an RSGraph, list or record of vertex labels, and list or record of arc labels)

▷ `LocalActionDiagramFromData(`*`graph, vertex_labels, arc_labels`*`)` (operation)
▷ `LocalActionDiagramFromDataNC(`*`graph, vertex_labels, arc_labels`*`)` (operation)

**Returns:** A local action diagram object.

This function constructs a local action diagram by having all of the diagrams data given to it. This is done by providing the `RSGraph` of the diagram and either lists of vertex and arc labels or records of vertex and arc labels.

If two lists are provided then the vertex and arc labels are assumed to be in the same order as `RSGraphVertices(`*`graph`*`)` and `RSGraphArcIDs(`*`graph`*`)` respectively. This means, for example, that element `idx` of *`vertex_labels`* is the label of the vertex with id `RSGraphVertexIDs(`*`graph`*`)[idx]`. If two records are provided then the names of the record elements are the vertex and arc ids respectively and the components are their labels.

The vertex labels are permutation groups. The domain each label acts on is stored in the mutable attribute `PermGroupDomain` (see `PermGroupDomain` (2.1.2)). By default, this is equal to the `MovedPoints` of the group but can be changed to allow for the group to have fixed points.

The arc labels are lists of integers. Each arc must be labelled by by an orbit of the vertex label at the vertex that arc originates at. Furthermore, every vertex must have an arc originating at it for each orbit of its vertex label. Note that the orbits of a vertex label `G` are determined by `Orbits(G, PermGroupDomain(G))`.

Unlike the formal definition of a local action diagram, we allow for overlap between the domains of different vertex labels. For example, a two vertex local action diagram could be labelled by $C_2$ and $S_3$ and they can have domains $\{1,2\}$ and $\{1,2,3\}$ respectively.

The NC variant of the function does not check that there is a label for each vertex and arc (and exactly this many labels), that the domains of the groups match up with the arc labels, or that the orbits of the vertex labels match up with the arc labels.

Example

```
gap> graph := RSGraphByAdjacencyList([[1, 1], [1, 2], [2, 1]], (2,3));;
gap> label_1 := Group((1,2));;
gap> SetPermGroupDomain(label_1, [1, 2, 3]);;
gap> label_2 := Group((1,2,3));;
gap> vertex_labels := [label_1, label_2];;
gap> arc_labels := [[1, 2], [3], [1,2,3]];;
gap> lad := LocalActionDiagramFromData(graph, vertex_labels, arc_labels);
<LocalActionDiagram with 2 vertices and 3 arcs>
gap> Print(lad);
```

```
Vertices = { 1, 2 }
Arcs = {
        1 = ( origin = 1, terminus = 1, inverse = 1 )
        2 = ( origin = 1, terminus = 2, inverse = 3 )
        3 = ( origin = 2, terminus = 1, inverse = 2 )
}
Reverse Map = (2,3)
Vertex Labels = {
        1 = Group( [ (1,2) ] )
        2 = Group( [ (1,2,3) ] )
}
Arc Labels = {
        1 = [ 1, 2 ]
        2 = [ 3 ]
        3 = [ 1, 2, 3 ]
}
```

### 2.1.4 LocalActionDiagramFromBurgerMozesUniversalGroup

▷ `LocalActionDiagramFromBurgerMozesUniversalGroup(`*perm_group*`)` (operation)

**Returns:** A local action diagram object.

Given a permutation group $F$ (*perm_group*) this function returns the local action diagram corresponding to the Burger-Mozes universal group $U(F)$ (see [cite Burger-Mozes and Colin and Simon for construction?]). This is a local action diagram with one vertex labelled by $F$ and a self-reverse loop for each orbit of $F$. Note that $F$ acts on the domain `PermGroupDomain(`*perm_group*`)`.

Example

```
gap> lad := LocalActionDiagramFromBurgerMozesUniversalGroup(Group((1,2)(3,4)));
<U(Group( [ (1,2)(3,4) ] )) (as a Local Action Diagram)>
```

### 2.1.5 LocalActionDiagramConstructBipartitionPreserving

▷ `LocalActionDiagramConstructBipartitionPreserving(`*lad*`)` (operation)

**Returns:** A local action diagram object.

Given a single vertex local action diagram $\Delta$ (*lad*) corresponding to the group $U(\Delta)$ acting on a tree $T$ this function returns the local action diagram corresponding to the group $U(\Delta)^\circ = \{g \in U(\Delta) \,|\, \forall v \in V(T), d(v,gv) \equiv 0 \pmod 2\}$. Intuitively this is the group of all automorphisms the preserve the bipartition of $T$ which always exists since a single vertex local action diagram is vertex transitive.

Given a group $G$ labelling the single vertex of $\Delta$ and arc labels $X_a$ $r$ this local action diagram is constructed as follows:

- Make a graph with two vertices and an edge for each loop of the graph for $\Delta$.
- Label both vertices by $G$.
- For each arc $a$ in the graph of $\Delta$ label the corresponding arc in the new graph by $X_a$ and the reverse of this arc by $X_{\bar{a}}$.

## 2.2 Local Action Diagram Attributes and Properties

### 2.2.1 LocalActionDiagramRSGraph

▷ `LocalActionDiagramRSGraph(`*lad*`)` (attribute)
**Returns:** RSGraph associated with the local action diagram.

### 2.2.2 LocalActionDiagramVertexLabels

▷ `LocalActionDiagramVertexLabels(`*lad*`)` (attribute)
**Returns:** Vertex labels of the local action diagram.

### 2.2.3 LocalActionDiagramArcLabels

▷ `LocalActionDiagramArcLabels(`*lad*`)` (attribute)
**Returns:** Arc labels of the local action diagram.

### 2.2.4 LocalActionDiagramVertices

▷ `LocalActionDiagramVertices(`*lad*`)` (attribute)
**Returns:** The vertices of the local action diagrams graph.
Shorthand for `RSGraphVertices(LocalActionDiagramRSGraph(`*lad*`))`.

### 2.2.5 LocalActionDiagramArcs

▷ `LocalActionDiagramArcs(`*lad*`)` (attribute)
**Returns:** The arcs of the local action diagrams graph.
Shorthand for `RSGraphArcs(LocalActionDiagramRSGraph(`*lad*`))`.

### 2.2.6 LocalActionDiagramArcIDs

▷ `LocalActionDiagramArcIDs(`*lad*`)` (attribute)
**Returns:** The arc ids of the local action diagrams graph.
Shorthand for `RSGraphArcIDs(LocalActionDiagramRSGraph(`*lad*`))`.

### 2.2.7 LocalActionDiagramNumberVertices

▷ `LocalActionDiagramNumberVertices(`*lad*`)` (attribute)
**Returns:** The number of vertices in the local action diagrams graph.
Shorthand for `RSGraphNumberVertices(LocalActionDiagramRSGraph(`*lad*`))`.

### 2.2.8 LocalActionDiagramNumberArcs

▷ `LocalActionDiagramNumberArcs(`*lad*`)` (attribute)
**Returns:** The number of arcs in the local action diagrams graph.
Shorthand for `RSGraphNumberArcs(LocalActionDiagramRSGraph(`*lad*`))`.

### 2.2.9 LocalActionDiagramReverseMap

▷ `LocalActionDiagramReverseMap(lad)` (attribute)

**Returns:** The reverse map of the local action diagrams graph.

Shorthand for `RSGraphReverseMap(LocalActionDiagramRSGraph(lad))`.

### 2.2.10 LocalActionDiagramOutNeighbours

▷ `LocalActionDiagramOutNeighbours(lad)` (attribute)

**Returns:** The record of out neighbours of the local action diagrams graph.

Shorthand for `RSGraphOutNeighbours(LocalActionDiagramRSGraph(lad))`.

### 2.2.11 LocalActionDiagramInNeighbours

▷ `LocalActionDiagramInNeighbours(lad)` (attribute)

**Returns:** The record of in neighbours of the local action diagrams graph.

Shorthand for `RSGraphInNeighbours(LocalActionDiagramRSGraph(lad))`.

### 2.2.12 LocalActionDiagramOutArcs

▷ `LocalActionDiagramOutArcs(lad)` (attribute)

**Returns:** The record of out arcs of the local action diagrams graph.

Shorthand for `RSGraphOutArcs(LocalActionDiagramRSGraph(lad))`.

### 2.2.13 LocalActionDiagramInArcs

▷ `LocalActionDiagramInArcs(lad)` (attribute)

**Returns:** The record of in arcs of the local action diagrams graph.

Shorthand for `RSGraphInArcs(LocalActionDiagramRSGraph(lad))`.

### 2.2.14 LocalActionDiagramGroupName

▷ `LocalActionDiagramGroupName(lad)` (attribute)

**Returns:** A name for the associated universal group of the local action diagram.

This is a mutable attribute which stores a human readable name for the associated universal group. By default it is equal to the empty string. In this case the `View` function will print `<LocalActionDiagram with n vertices and m arcs>` where `n` is the number of vertices and `m` is the number of arcs. If this attribute is set to to any other string then the `View` function will print `<{group_name} (as a Local Action Diagram)>`.

Example

```
gap> graph := RSGraphByAdjacencyList([[1, 1]], ());;
gap> vertex_labels := [SymmetricGroup(3)];;
gap> arc_labels := [[1,2,3]];;
gap> lad := LocalActionDiagramFromData(graph, vertex_labels, arc_labels);
<LocalActionDiagram with 1 vertex and 1 arc>
gap> SetLocalActionDiagramGroupName(lad, "Aut(T_3)");
gap> View(lad);
<Aut(T_3) (as a Local Action Diagram)>
```

### 2.2.15 LocalActionDiagramRegularTree

▷ `LocalActionDiagramRegularTree(lad)` (attribute)

**Returns:** The degree of the regular tree the universal group acts on or `fail` if it does not act on one.

If every vertex label of the local action diagram has a domain of size $d$ then the associated universal group acts on a regular tree of degree $d$. This functions returns $d$ if this is the case and `fail` otherwise.

Example

```
gap> graph := RSGraphByAdjacencyList([[1, 2], [2, 1]], (1,2));;
gap> vertex_labels := [SymmetricGroup(3), Group((1,2,3))];;
gap> arc_labels := [[1,2,3], [1,2,3]];;
gap> lad := LocalActionDiagramFromData(graph, vertex_labels, arc_labels);;
gap> LocalActionDiagramRegularTree(lad);
3
gap> vertex_labels_2 := [SymmetricGroup(3), Group((1,2))];;
gap> arc_labels_2 := [[1,2,3], [1,2]];;
gap> lad_2 := LocalActionDiagramFromData(graph, vertex_labels_2, arc_labels_2);;
gap> LocalActionDiagramRegularTree(lad_2);
fail
```

### 2.2.16 LocalActionDiagramScopos

▷ `LocalActionDiagramScopos(lad)` (attribute)

**Returns:** The list of scopos of the local action diagram.

(CITE REID SMITH HERE) Let $\Delta = (\Gamma, (X_a), (G(v)))$ be a local action diagram. A strongly confluent partial orientation (*scopo*) of $\Delta$ is a subset $O$ of $A(\Gamma)$ such that:

- if $a \in O$ then $\overline{a} \notin O$,
- if $a \in O$ then $|X_a| = 1$, and
- for all $v \in V(\Gamma)$ if $O$ contains an arc $a$ originating at $v$ then $O$ contains all arcs that terminate at $v$ except for $\overline{a}$.

This function performs an iterative search for all scopos of a local action diagram. It stores each scopo as a list of arc ids in the scopo. Note that the empty set is a scopo and this is represented as the empty list.

Example

```
gap> graph := RSGraphByAdjacencyList ([[1, 2], [2, 1]], (1,2));;
gap> group := Group(());;
gap> SetPermGroupDomain(group, [1]);;
gap> vertex_labels := [group, group];;
gap> arc_labels := [[1], [1]];;
gap> lad := LocalActionDiagramFromData(graph, vertex_labels, arc_labels);;
gap> LocalActionDiagramScopos(lad);
[ [  ], [ 1 ], [ 2 ] ]
```

### 2.2.17 LocalActionDiagramGroupType

▷ `LocalActionDiagramGroupType(`*lad*`)` (attribute)

**Returns:** The type of the corresponding universal group.

(CITE REID SMITH HERE) Every group acting on a tree can be split into one of six mutually exclusive types: *fixed vertex*, *edge inversion*, *lineal*, *horocyclic*, *focal*, and *general*. This types can be recognised from the groups corresponding local action diagram by analysing the scopos of the local action diagram (see CITE REID SMITH).

This function first calculates the scopos of the local action diagram (see `LocalActionDiagramScopos` (2.2.16)) and then uses this to determine the type of the corresponding group. It outputs this types as a string.

Example

```
gap> graph := RSGraphByAdjacencyList ([[1, 2], [2, 1]], (1,2));;
gap> group := Group(());;
gap> SetPermGroupDomain(group, [1]);;
gap> vertex_labels := [group, group];;
gap> arc_labels := [[1], [1]];;
gap> lad := LocalActionDiagramFromData(graph, vertex_labels, arc_labels);;
gap> LocalActionDiagramGroupType(lad);
"General"
```

### 2.2.18 LocalActionDiagramIsDiscrete

▷ `LocalActionDiagramIsDiscrete(`*lad*`)` (property)

**Returns:** `true` if the corresponding universal group is discrete and false otherwise.

This function implements (CITE THEOREM HERE) to determine if the corresponding universal group is discrete. This requires calculation of the diagrams group type (see `LocalActionDiagramGroupType` (2.2.17)). Furthermore, if the group is discrete then it is also uniscalar and unimodular and so if this function returns `true` then it also sets `LocalActionDiagramIsUniscalar` (2.2.19) and `LocalActionDiagramIsUnimodular` (2.2.20) to true.

Example

```
gap> graph := RSGraphByAdjacencyList ([[1, 2], [2, 1]], (1,2));;
gap> group := Group(());;
gap> SetPermGroupDomain(group, [1]);;
gap> vertex_labels := [group, group];;
gap> arc_labels := [[1], [1]];;
gap> lad := LocalActionDiagramFromData(graph, vertex_labels, arc_labels);;
gap> LocalActionDiagramIsDiscrete(lad);
true
```

### 2.2.19 LocalActionDiagramIsUniscalar

▷ `LocalActionDiagramIsUniscalar(`*lad*`)` (property)

**Returns:** `true` if the corresponding universal group is uniscalar and false otherwise.

This function implements (CITE THEOREM HERE) to determine if the corresponding universal group is uniscalar. This requires calculation of the diagrams group type (see `LocalActionDiagramGroupType` (2.2.17)). Furthermore, if the group is uniscalar then it is also

unimodular so if this function returns `true` then it also sets `LocalActionDiagramIsUnimodular` (2.2.20) to true.

Example

```
gap> graph := RSGraphByAdjacencyList ([[1, 2], [2, 1], [2, 2]], (1,2));;
gap> group := Group(());;
gap> group_2 := Group((1, 2), (1,2,3), (4,5));;
gap> SetPermGroupDomain(group, [1]);;
gap> vertex_labels := [group, group_2];;
gap> arc_labels := [[1], [1, 2, 3], [4, 5]];;
gap> lad := LocalActionDiagramFromData(graph, vertex_labels, arc_labels);;
gap> LocalActionDiagramIsDiscrete(lad);
false
gap> LocalActionDiagramIsUniscalar(lad);
true
```

### 2.2.20 LocalActionDiagramIsUnimodular

▷ `LocalActionDiagramIsUnimodular(`*lad*`)` (property)

**Returns:** `true` if the corresponding universal group is unimodular and false otherwise.

This function implements (CITE THEOREM HERE both us and BK) to determine if the corresponding universal group is unimodular. This requires calculation of a spanning tree of the graph (see `RSGraphSpanningTree` (1.3.4)).

Example

```
gap> adj_list := [[1, 2], [2, 1], [2, 3], [3, 2], [3, 1], [1, 3], [1, 1]];;
gap> graph := RSGraphByAdjacencyList(adj_list, (1,2)(3,4)(5,6));;
gap> vertex_labels := [];;
gap> Add(vertex_labels, Group((1, 2, 3), (4,5), (6,7)));;
gap> Add(vertex_labels, Group((1, 2), (3, 4, 5)));;
gap> Add(vertex_labels, Group((1, 2), (3, 4)));;
gap> arc_labels := [[1, 2, 3], [3, 4, 5], [1, 2], [1, 2], [3,4], [4,5], [6,7]];;
gap> lad := LocalActionDiagramFromData(graph, vertex_labels, arc_labels);;
gap> LocalActionDiagramIsDiscrete(lad);
false
gap> LocalActionDiagramIsUniscalar(lad);
false
gap> LocalActionDiagramIsUnimodular(lad);
true
```

### 2.2.21 LocalActionDiagramIsBurgerMozesUniversalGroup

▷ `LocalActionDiagramIsBurgerMozesUniversalGroup(`*lad*`)` (property)

**Returns:** `true` if the corresponding universal group is a Burger-Mozes Universal Group and `false` otherwise.

If a local action diagram has a single vertex and all loops at the vertex are self-reverse then the diagram corresponds to a Burger-Mozes group (see PROBABLY SECOND COLIN SIMON PAPER). If the given diagram has such a structure then this function returns true. It also sets `LocalActionDiagramGroupName` (2.2.14) to `"U({vertex_label})"` where `{vertex_label}` is the name of the group labelling the vertex.

Example

```
gap> graph := RSGraphByAdjacencyList([[1, 1], [1, 1]], ());;
gap> vertex_labels := [Group((1, 2), (3, 4))];;
gap> SetName(vertex_labels[1], "C2 x C2");
gap> arc_labels := [[1, 2], [3, 4]];;
gap> lad := LocalActionDiagramFromData(graph, vertex_labels, arc_labels);;
gap> View(lad);
<LocalActionDiagram with 1 vertex and 2 arcs>
gap> LocalActionDiagramIsBurgerMozesUniversalGroup(lad);
true
gap> View(lad);
<U(C2 x C2) (as a Local Action Diagram)>
```

### 2.2.22 LocalActionDiagramIsSmithUniversalGroup

▷ `LocalActionDiagramIsSmithUniversalGroup(`*lad*`)` (property)

**Returns:** `true` of the local action diagram corresponds to a smith universal group and `false` otherwise.

This function checks if the local action diagram corresponds to a box product universal group (CITE SMITH PAPER?). Such a local action diagram consists of a bipartite graph such that vertex labels in the same partition are conjugate (see (CITE COLIN-SIMON INTRO PAPER)). If the diagram has this structure, this function returns `true` and sets `LocalActionDiagramGroupName` (2.2.14) to `U({G1}, {G2})` where `G1` and `G2` are one of the vertex labels in each partition.

Note that this function requires the Digraphs to be loaded as it relies on the isomorphism functions (see `Isomorphisms and Automorphisms` (4)).

### 2.2.23 LocalActionDiagramIsEndStabiliser

▷ `LocalActionDiagramIsEndStabiliser(`*lad*`)` (property)

**Returns:** `true` if the corresponding universal group is the stabiliser of a single end in the full automorphism group of a regular tree and `false` otherwise. otherwise.

If a local action diagram has a single vertex with two mutually reverse loops and the vertex is labelled by $S_{d-1}$ acting on $\{1,2,\ldots,d\}$ then the universal group is isomorphic to $\mathrm{Aut}(T_d)_\omega$ where $\omega$ is an end of $T_d$. This function returns `true` if the group satisfies this structure and `false` otherwise. It also sets `LocalActionDiagramGroupName` (2.2.14) to `Aut(Td)_omega`.

Example

```
gap> graph := RSGraphByAdjacencyList([[1, 1], [1, 1]], (1, 2));;
gap> vertex_labels := [SymmetricGroup(2)];;
gap> SetPermGroupDomain(vertex_labels[1], [1,2,3]);;
gap> arc_labels := [[1, 2], [3]];;
gap> lad := LocalActionDiagramFromData(graph, vertex_labels, arc_labels);;
gap> View(lad);
<LocalActionDiagram with 1 vertex and 2 arcs>
gap> LocalActionDiagramIsEndStabiliser(lad);
true
gap> View(lad);
<Aut(T3)_omega (as a Local Action Diagram)>
```

### 2.2.24 LocalActionDiagramIsRegularTreeAutomorphismGroup

▷ `LocalActionDiagramIsRegularTreeAutomorphismGroup(`*lad*`)` (property)

**Returns:** `true` if the corresponding universal group is the full automorphism group of a regular tree and `false` otherwise.

If a local action diagram has a single vertex with one self-reverse loop and the vertex is labelled by $S_d$ then the universal group is isomorphic to $\mathrm{Aut}(T_d)$. This function returns `true` if the group satisfies this structure and `false` otherwise. It also sets `LocalActionDiagramGroupName` (2.2.14) to `Aut(Td)`.

Example

```
gap> graph := RSGraphByAdjacencyList([[1, 1]], ());;
gap> vertex_labels := [SymmetricGroup(5)];;
gap> arc_labels := [[1..5]];;
gap> lad := LocalActionDiagramFromData(graph, vertex_labels, arc_labels);;
gap> View(lad);
<LocalActionDiagram with 1 vertex and 1 arc>
gap> LocalActionDiagramIsRegularTreeAutomorphismGroup(lad);
true
gap> View(lad);
<Aut(T5) (as a Local Action Diagram)>
```

### 2.2.25 LocalActionDiagramIsBipartitionPreservingGroup

▷ `LocalActionDiagramIsBipartitionPreservingGroup(`*lad*`)` (property)

**Returns:** `true` of the local action diagram corresponds to a bipartition preserving group and `false` otherwise.

This function checks if the local action diagram corresponds to a bipartition preserving group group. Given a vertex transitive group $G$ acting on a regular tree $T$ this group is the subgroup $G^\circ = \{g \in G : \forall v \in V(T)\, d(v,gv) \equiv 0 \pmod 2\}$. In words, its the maximal subgroup which partitions the tree into two vertex orbits. If the diagram has this structure, this function returns `true` and sets `LocalActionDiagramGroupName` (2.2.14) to `{base_name}^o` where `{base_name}` is the name of the underlying vertex transitive group.

Note that this function requires the Digraphs to be loaded as it relies on the isomorphism functions (see `Isomorphisms and Automorphisms` (4)).

# Chapter 3

# IO Operations and Visualisation

## 3.1 IO Operations

### 3.1.1 Writing To A String

▷ `RSGraphToWritableString(`*graph*`)` (operation)
▷ `LocalActionDiagramToWritableString(`*lad*`)` (operation)

**Returns:** A String representation of the object.

These two functions turn an RSGraph or local action diagram into a string representation. The exact format of the representations are described in (APPENDIX REFERENCE). These functions are not designed to store the objects in the most space-efficient format. The formats are designed so that the objects can be read into a GAP session with very little overhead and so that the objects and known attributes can be recreated in a new GAP session.

Note that these functions do not perform any disk IO operations themselves. They return a string in the GAP session and it is up to the user to chose how disk IO operations with the string are done.

For an RSGraph the data that is always stored is:

- the vertex ids,
- the arc ids,
- the adjacency list, and
- the reverse map.

If the canonical labelling or automorphism group (REF FUNCTIONS) of the RSGraph are known then they are also stored.

For a local action diagram that data that is always stored is:

- the diagrams RSGraph (and all data known about it),
- the vertex labels, and
- the arc labels.

The data stored if it is known is:

- the diagrams scopos,

- the corresponding group type,
- if the corresponding group is discrete,
- if the corresponding group is uniscalar,
- if the corresponding group is unimodular, and
- the name of the corresponding group.

Example

```
gap> graph := RSGraphByAdjacencyList([[1, 1]], ());;
gap> RSGraphToWritableString(graph);
"1|1,1,1|P"
gap> lad := LocalActionDiagramFromData(graph, [Group((1,2))], [[1,2]]);;
gap> LocalActionDiagramGroupType(lad);;
gap> LocalActionDiagramToWritableString(lad);
"1|1,1,1|P/1:P2,1:1,2|1:1,2|LocalActionDiagramGroupType!General|LocalActionDia\
gramScopos! "
```

### 3.1.2 Reading From A String

▷ RSGraphFromWritableString(*string*) (operation)

▷ LocalActionDiagramFromWritableString(*string[, return_graph]*) (operation)

**Returns:** The object(s) the string represents.

These function take the output from the RSGraphToWritableString and LocalActionDiagramToWritableString" methods as input and returns the objects those strings represents. While it is possible to edit these strings outside of GAP these functions do not perform error checking and so this can potentially lead to invalid RSGraph or local action diagram objects. If the optional *return_graph* parameter is set to true then the LocalActionDiagramFromWritableString function will return a list whose first entry is the local action diagram object and second intro is the RSGraph for this local action diagram.

Any optional attributes known about the objects at the time of writing will be stored in the objects returned by these functions without needing to be recomputed. Note that the (REF canon and aut functions) attributes can be stored in the objects created by these functions without needing the Digraphs to be loaded even though computing these attributes required the Digraphs to be loaded.

Example

```
gap> graph_string := "1|1,1,1|P";;
gap> graph := RSGraphFromWritableString(graph_string);;
gap> Print(graph);
Vertices = { 1 }
Arcs = {
        1 = ( origin = 1, terminus = 1, inverse = 1 )
}
Reverse Map = ()
gap> lad_string := "1|1,1,1|P/1:P2,1:1,2|1:1,2|LocalActionDiagramGroupType!Gene\
> ral|LocalActionDiagramScopos! ";;
gap> lad := LocalActionDiagramFromWritableString(lad_string);;
gap> Print(lad);
Vertices = { 1 }
Arcs = {
```

```
        1 = ( origin = 1, terminus = 1, inverse = 1 )
}
Reverse Map = ()
Vertex Labels = {
        1 = Group( [ (1,2) ] )
}
Arc Labels = {
        1 = [ 1, 2 ]
}
gap> Print("LocalActionDiagramScopos" in KnownAttributesOfObject(lad));
true
gap> Print("LocalActionDiagramIsDiscrete" in KnownAttributesOfObject(lad));
false
```

### 3.1.3 IO Pickling/Unpickling

▷ IO_Pickle(*[file, ]graph*) (operation)

▷ IO_Pickle(*[file, ]lad*) (operation)

**Returns:** IO_OK or the pickle string if the pickling was successful or IO_ERROR if there was an error.

▷ IO_Unpickle(*file*) (operation)

**Returns:** An RSGraph or Local Action Diagram object if successful or IO_ERROR if there was an error.

The IO package supports serialising data through the IO_Pickle function. Unlike the RSGraphToWritableString and LocalActionDiagramToWritableString functions these functions support recreating an arbitrary structure within the memory of a GAP session (assuming that the pickling functions have been defined for each object). In particular, it can easily recreate data structures storing objects.

For example, if l is a list of RSGraphs then you can write this list to the disk with a single call to IO_Pickle. This generality comes at the cost of performance. For this reason, if you have a large collection of RSGraph and local action diagram objects that you need writing to the disk we recommend using the RSGraphToWritableString and LocalActionDiagramToWritableString functions.

In the one argument version for IO_Pickle, the pickle string for the object is returned if the pickling was successful. In the two argument version, an IO file opened to write mode is supplied and the pickle string is written to this file.

The IO_Unpickle function accepts either the pickle string as input or an IO file opened to read mode which stores pickled objects. It recreates the objects in memory exactly.

As a note for developers, the RSGraph and local action diagram objects are pickled with the "magic values" RSGO and LADO respectively.

Example

```
gap> graph := RSGraphByAdjacencyList([[1, 1]], ());;
gap> lad := LocalActionDiagramFromData(graph, [Group((1,2))], [[1,2]]);;
gap> list := [graph, lad];
[ <RSGraph with 1 vertex and 1 arc>,
  <LocalActionDiagram with 1 vertex and 1 arc> ]
gap> file := IO_File("test.pickle", "w");;
gap> IO_Pickle(file, list);
IO_OK
```

```
gap> IO_Close(file);;
gap> file := IO_File("test.pickle", "r");;
gap> IO_Unpickle(file);
[ <RSGraph with 1 vertex and 1 arc>,
  <LocalActionDiagram with 1 vertex and 1 arc> ]
```

## 3.2 Visualisation

*Stephan to do.*

# Chapter 4

# Isomorphisms and Automorphisms

We support calculating isomorphisms between RSGraphs, calculating the automorphism group of an RSGraph, and calculating isomorphisms between local action diagrams. This functionality relies on the isomorphism and automorphism functions from the Digraphs and so is not available if the Digraphs is not available. By default this makes use of `Bliss` for calculating morphisms but it can use `Nauty` if the NautyTracesInterface is available.

Given two RSGraphs $\Gamma_1$ and $\Gamma_2$ an *isomorphism* is a pair of bijective maps $\Theta_V : V(\Gamma_1) \to V(\Gamma_2)$ and $\Theta_A : A(\Gamma_1) \to A(\Gamma_2)$ that respect origin vertices and the arc reversal map. Explicitly, this means that $\Theta_V(o(a)) = o(\Theta_A(a))$ and $\overline{\Theta_A(a)} = \Theta_A(\overline{a})$. An automorphism is an isomorphism from an RSGraph to itself and the set of all automorphism of an RSGraph form a group under function composition.

Given two local action diagrams $\Delta = (\Gamma, (X_a), (G(v)))$ and $\Delta' = (\Gamma', (X'_a), (G'(v)))$ and isomorphism between $\Delta$ and $\Delta'$ consists of: - an isomorphism $\Theta : \Gamma \to \Gamma'$ and - for each vertex $v \in V(\Gamma)$ a bijection $\Theta_v : X_v \to X_{\Theta(v)}$ that restricts to a bijection from $X_a$ to $X_{\Theta(a)}$ for each $a \in o^{-1}(a)$ and such that $\Theta_v G(v) \Theta_v^{-1} = G'(v)$.

## 4.1 RSGraph Morphisms

### 4.1.1 AutomorphismGroup (for an RSGraph)

▷ `AutomorphismGroup(`*graph*`)` (operation)

**Returns:** A permutation group.

This function returns the automorphism group of the *graph* as a permutation group. Each permutation in the group is an automorphism on the arcs of the graph. For each arc automorphism the corresponding vertex automorphism can be found with the `RSGraphVertexAutomorphism` (4.1.3) function.

Example

```
gap> graph := RSGraphByAdjacencyList([[1, 1], [1, 1], [1, 1]], (2,3));;
gap> AutomorphismGroup(graph);
Group([ (2,3) ])
```

### 4.1.2 IsomorphismRSGraphs

▷ `IsomorphismRSGraphs(`*graph_1, graph_2*`)` (operation)

**Returns:** A permutation or general mapping if the graphs are isomorphic and `fail` otherwise.

If *graph_1* and *graph_2* are isomorphic then this function returns one arc isomorphism between them. If the graphs have the same arc ids then this isomorphism is returned as a permutation. If they have different arc ids then the isomorphism is returned as a constant access time general mapping.

This isomorphism found is not necessarily unique. To get every isomorphism you can find the automorphism group of one of the graphs and compose these automorphisms with the isomorphism found (see `RSGraphsIsomorphismsIterator` (4.1.5). For each arc isomorphism you can get the corresponding vertex isomorphism with the `RSGraphsVertexIsomorphism` (4.1.4) function.

The isomorphisms are found by finding a "canonical form" of each graph (see `RSGraphCanonicalLabelling` (4.1.6)) and composing the mappings to the canonical form. This means that subsequent calls of this function on the same graphs have constant time complexity.

Example

```
gap> graph_1 := RSGraphByAdjacencyList([[1, 1], [1, 1], [1, 1]], (2,3));;
gap> graph_2 := RSGraphByAdjacencyList([[1, 1], [1, 1], [1, 1]], (1,2));;
gap> IsomorphismRSGraphs(graph_1, graph_2);
(1,3,2)
```

### 4.1.3 RSGraphVertexAutomorphism

▷ `RSGraphVertexAutomorphism(`*graph, arc_aut*`)` (operation)

**Returns:** A permutation.

Given an RSGraph and an automorphism of the arcs this function returns the corresponding automorphism of the vertices of the graph. The automorphism of the arcs is represented as a permutation. If the argument *arc_aut* is not an automorphism of the graph's arcs then the behaviour of this function is undefined.

Example

```
gap> graph := RSGraphByAdjacencyList([[1, 1], [1, 1], [1, 2], [2, 1], [2, 2], [2, 2]], (3,4));;
gap> aut_group := AutomorphismGroup(graph);;
gap> arc_aut := Elements(aut_group)[5];
(1,5)(2,6)(3,4)
gap> vert_aut := RSGraphVertexAutomorphism(graph, arc_aut);
(1,2)
```

### 4.1.4 RSGraphsVertexIsomorphism

▷ `RSGraphsVertexIsomorphism(`*graph_1, graph_2, arc_iso*`)` (operation)

**Returns:** A permutation or general mapping.

Given two RSGraphs and an isomorphism of the arcs between them this function returns the corresponding isomorphism between the vertices of the graphs. The argument *arc_iso* can either be a permutation or a general mapping. If the vertex ids of the two graphs are the same then the isomorphism will be returned as a permutation. Otherwise it will be returned as a general mapping. If the argument *arc_iso* is not an isomorphism between the arcs of the two graphs then the behaviour of this function is undefined.

Example

```
gap> graph_1 := RSGraphByAdjacencyList([[1, 1], [1, 1], [1, 1]], (2,3));;
gap> graph_2 := RSGraphByAdjacencyList([[2, 2], [2, 2], [2, 2]], (1,2), [2]);;
gap> arc_iso := IsomorphismRSGraphs(graph_1, graph_2);
(1,3,2)
gap> vert_iso := RSGraphsVertexIsomorphism(graph_1, graph_2, arc_iso);
```

```
<general mapping: Domain([ 1 ]) -> Domain([ 2 ]) >
gap> 1^vert_iso;
2
```

### 4.1.5 RSGraphsIsomorphismsIterator

▷ `RSGraphsIsomorphismsIterator(`*graph_1, graph_2*`)` (operation)

**Returns:** An iterator over all permutations between two RSGraphs.

This function returns an iterator over all isomorphisms between *graph_1* and *graph_2* composing every automorphism of *graph_1* with a "base isomorphism" between *graph_1* and *graph_2*. Note that as the automorphism group is a stored attribute of each graph if the same graph is used for multiple calls of this function it should be used as the first argument to avoid calculating the automorphism group each time.

The iterator returns the isomorphisms in the form `[vertex_iso, arc_iso]`. The elements of this list will be either permutations are general mappings depending on whether or not the vertex/arc ids are the same between both graphs.

Example

```
gap> graph_1 := RSGraphByAdjacencyList([[1, 1], [1, 1], [1, 1]], ());;
gap> graph_2 := RSGraphByAdjacencyList([[1, 1], [1, 1], [1, 1]], ());;
gap> for iso in RSGraphsIsomorphismsIterator(graph_1, graph_2) do
>            Print(iso, "\n");
>    od;
[ (), () ]
[ (), (2,3) ]
[ (), (1,3,2) ]
[ (), (1,3) ]
[ (), (1,2,3) ]
[ (), (1,2) ]
```

### 4.1.6 RSGraphCanonicalLabelling

▷ `RSGraphCanonicalLabelling(`*graph*`)` (operation)

**Returns:** A record to to identify the "canonical labelling" of the graph.

Given a graph $\Gamma_1$ a canonical form of $\Gamma_1$ is a graph $\rho(\Gamma_1)$ that is isomorphic to $\Gamma_1$ and such that every graph isomorphic to $\Gamma_1$ has Canonical form $\rho(\Gamma_1)$. This means that $\Gamma_1$ and $\Gamma_2$ are isomorphic if and only if $\rho(\Gamma_1) = \rho(\Gamma_2)$. Finding canonical labellings of a graph is how both *Bliss* and *Nauty* find graph isomorphisms.

This function returns a record containing all information of the "canonical labelling" of the graph. The elements of the record are:

- `canon_certificate`: A string that represents the canonical form of the graph. Two graphs are isomorphic if and only if this string has the same value for both.
- `arc_isomorphism`: This maps the arcs of the graph to the arcs of the canonical form of the graph. It does not take into account the reverse map.
- `vertex_isomorphism`: This maps the vertices of the graph to the vertices of the canonical form of the graph.

- `arc_standard_map`: This maps the arcs of the graph so that loops that are not self-reverse have smaller arc ids then self-reverse loops at the same vertex. This is needed to make sure the reverse map of the canonical forms for two graphs is the same if the graphs are isomorphic.
- `reverse_map`: The reverse map of the canonical form of the graph.

Using this information it is possible to construct an isomorphism between two graphs by following the maps from the first graph to the canonical form and then taking the inverses maps to the second graph. This is how the `IsomorphismRSGraphs` (4.1.2) function works.

Example

```
gap> graph_1 := RSGraphByAdjacencyList([[1, 1], [1, 1], [1, 1]], (1,2));;
gap> graph_2 := RSGraphByAdjacencyList([[1, 1], [1, 1], [1, 1]], (2,3));;
gap> canon_1 := RSGraphCanonicalLabelling(graph_1);;
gap> canon_2 := RSGraphCanonicalLabelling(graph_2);;
gap> Print(canon_1);
rec(
  arc_isomorphism := (),
  arc_standard_map := (),
  canon_certificate := "[ [ 3 ] ]2;",
  reverse_map := (1,2),
  vertex_isomorphism := () )
gap> graph_iso := canon_1.arc_isomorphism;;
gap> graph_iso := graph_iso*canon_1.arc_standard_map;;
gap> graph_iso := graph_iso*Inverse(canon_2.arc_standard_map);;
gap> graph_iso := graph_iso*Inverse(canon_2.arc_isomorphism);;
gap> Print(graph_iso);
(1,2,3)
gap> Print(IsomorphismRSGraphs(graph_1, graph_2));
(1,2,3)
```

### 4.1.7 IsomorphismLocalActionDiagrams

▷ `IsomorphismLocalActionDiagrams(`*`lad_1, lad_2`*`)` (operation)

**Returns:** A list containing the vertex and arc isomorphisms and record of bijective maps or `fail` if there is no isomorphism.

This function searches for a local action diagram isomorphism between two local action diagrams. It does this by iterating over every graph isomorphism between the two underlying graphs. It then searches through every possible bijection between the vertex label's domains based on where the vertices map to under the isomorphism. This search checks if the vertex labels are conjugate under each bijection. If it can find one for each vertex label then it has found a local action diagram isomorphism. If it can't and has exhausted all graph isomorphisms then the local action diagrams are not isomorphic.

If the diagrams are isomorphic then this function returns the isomorphism found in the form `[vert_iso, arc_iso, bijection_rec]`. The first two entries of this list are either permutations are bijections (see `IsomorphismRSGraphs` (4.1.2)). The final entry is a record where the names are the vertex ids of *`lad_1`*. Each component contains the bijective map between the two domains which conjugates the groups. Specifically, if `i` is a vertex label of *`lad_1`* then `bijection_rec.(i)` is a mapping from the domain of the group labelling vertex `i` in *`lad_1`* to the domain of the group labelling `i^vert_iso` in *`lad_2`*.

Example

```
gap> graph := RSGraphByAdjacencyList([[1, 2], [2, 1]], (1,2));;
gap> lad_1 := LocalActionDiagramFromData(graph, [Group((1,2)), Group((3,4))], \
>                                        [[1, 2], [3, 4]]);;
gap> lad_2 := LocalActionDiagramFromData(graph, [Group((3,4)), Group((1,2))], \
>                                        [[3, 4], [1, 2]]);;
gap> iso := IsomorphismLocalActionDiagrams(lad_1, lad_2);;
gap> for elm in iso do Print(elm, "\n"); od;
()
()
rec(
  1 := <mapping: Domain([ 1, 2 ]) -> Domain([ 3, 4 ]) >,
  2 := <mapping: Domain([ 3, 4 ]) -> Domain([ 1, 2 ]) > )
```

# Chapter 5

# Enumeration

Since there is a one-to-one correspondence between isomorphism classes of local action diagrams and conjugacy classes Of (P)-closed groups enumerating local action diagrams is the same as enumerating (P)-closed groups. By enumerating local action diagrams you can, for example, find the number of conjugacy classes of (P)-closed groups acting on a certain tree with a specified number of orbits. You can also find how many of these groups satisfy other properties, such as unimodularity.

We have developed algorithms to enumerate local action diagrams acting on a regular tree with a specified number of orbits. For the inputs we could run these algorithms on we have stored the results so they can be quickly accessed. Furthermore, we have implemented functions analogous to the Transitive Groups package for easily filtering the library of local action diagrams.

## 5.1 Enumerating RSGraphs and Local Action Diagrams

### 5.1.1 RSGraphFromLibrary

▷ `RSGraphFromLibrary(`*`degree, no_verts[, idx]`*`)` (operation)

**Returns:** A list of RSGraphs or a specific element from this list.

This function returns a list of RSGraphs such that each vertex has degree less than or equal to *`degree`* and the graph has exactly *`no_verts`* vertices. Each graph in the list is unique up to isomorphism. If the optional argument *`idx`* is provided then it will return the RSGraph in that position of the list. This is analogous to the `TransitiveGroup` function from the Transitive Groups package.

There are two ways this function can get the list of RSGraphs. If the data for the given degree and number of vertices is stored on the disk then it will be read from the disk. If the data is not stored on the disk then the RSGraph enumeration algorithm will be run instead. An information message is displayed if the enumeration algorithm is used.

Note that the Digraphs is required for the enumeration algorithm to run. If it is not loaded then the function will return an error if it needs to run the enumeration algorithm. The Digraphs is NOT needed if the data is stored on the disk.

Example

```
gap> graph_list := RSGraphFromLibrary(3, 2);;
gap> graph := graph_list[4];
<RSGraph with 2 vertices and 3 arcs>
gap> graph_2 := RSGraphFromLibrary(3, 2, 4);;
gap> IsomorphismRSGraphs(graph, graph_2);
()
```

### 5.1.2 LocalActionDiagramFromLibrary

▷ LocalActionDiagramFromLibrary(*degree, no_verts[, idx]*) (operation)

**Returns:** A list of local action diagrams or a specific element from this list.

This function returns a list of local action diagrams such that each vertex label has degree equal to *degree* and the graph has exactly *no_verts* vertices. Each local action diagram in the list is unique up to isomorphism. If the optional argument *idx* is provided then it will return the local action diagram in that position of the list. This is analogous to the TransitiveGroup function from the Transitive Groups package.

There are two ways this function can get the list of local action diagrams. If the data for the given degree and number of vertices is stored on the disk then it will be read from the disk. If the data is not stored on the disk then the local action diagram enumeration algorithm will be run instead. An information message is displayed if the enumeration algorithm is used.

Note that the Digraphs is required for the enumeration algorithm to run. If it is not loaded then the function will return an error if it needs to run the enumeration algorithm. The Digraphs is NOT needed if the data is stored on the disk.

Example

```
gap> lad_list := LocalActionDiagramFromLibrary(3, 2);;
gap> lad := lad_list[4];
<LocalActionDiagram with 2 vertices and 4 arcs>
gap> lad_2 := LocalActionDiagramFromLibrary(3, 2, 4);;
gap> IsomorphismLocalActionDiagrams(lad, lad_2);
[ (), (), rec( 1 := <mapping: Domain([ 1, 2, 3 ]) -> Domain([ 1, 2, 3 ]) >,
      2 := <mapping: Domain([ 4, 5, 6 ]) -> Domain([ 4, 5, 6 ]) > ) ]
```

### 5.1.3 Number of RSGraphs/Local Action Diagrams

▷ NumberRSGraphs(*degree, no_verts*) (operation)

▷ NumberLocalActionDiagrams(*degree, no_verts*) (operation)

**Returns:** The number of RSGraphs or Local Action Diagrams with a specified degree and number of vertices.

These functions returns the number of RSGraphs or Local Action Diagrams with degree *degree* and *no_verts* vertices. They are a shorthand for Size(RSGraphFromLibrary(*degree*, *no_verts*) and Size(LocalActionDiagramFromLibrary(*degree*, *no_verts*). They are included to be analogous to the NrTransitiveGroups function from th Transitive Groups library.

Example

```
gap> lad_list := LocalActionDiagramFromLibrary(3, 3);;
gap> Size(lad_list);
78
gap> NumberLocalActionDiagrams(3, 3);
78
```

## 5.2 Searching The Library

### 5.2.1 AllRSGraphs/AllLocalActionDiagrams

▷ AllRSGraphs(*[function_1, value_1, ...]*) (operation)

▷ AllLocalActionDiagrams(*[function_1, value_1, ...]*) (operation)

**Returns:** All RSGraphs or Local Action Diagrams satisfying the conditions given as arguments.

These functions return a list of RSGraphs or Local Action Diagrams satisfying the conditions specified. The functions take an even number of inputs which alternate between functions and values. Each function must have a single argument as input which is either an RSGraph or local action diagram. The output of *function_i* must be comparable with the equality operation to *value_i*. The functions are analogous to the AllTransitiveGroups function from the Transitive Groups package.

These functions work by first starting with a list of every RSGraph or Local Action Diagram stored on the disk. Each function supplied as an argument is applied to every element of this list and the result of applying this function is compared with the provided value. If it matches the value then the element is kept in the list; otherwise it is discarded. After this process is done for every argument the final list is returned. Note that it can be an empty list if there are no elements satisfying all conditions.

If any of the functions provided are properties or attribute of RSGraphs or Local Action Diagrams then these functions are prioritised for evaluation. This means that any function which is not a property or attribute will be evaluated after each property and attribute is. The reason for this is that properties and attributes Are known the by "easy" to calculate or are stored in the library data and so this can speed up the search by reducing the search space for the potentially "harder" functions.

The first time one of these functions is called the entire library is read into the GAP sessions memory. Information messages are displayed during this process as this can take some time. Subsequent calls to this function do not need to read the library files.

If no arguments are provided then a complete list of RSGraphs or local action diagrams is returned. Furthermore, every degree two RSGraph and local action diagram can be quickly manually constructed. By default, this function only includes those with up to ten vertices. This can be changed with the SetLocalActionDiagramDegreeTwoVertexBound (5.2.3) function.

Example

```
gap> TestFunction := function(lad)
>            return LocalActionDiagramVertexLabels(lad).(1);
>    end;
function( lad ) ... end
gap> lad_list := AllLocalActionDiagrams(TestFunction, Group((1,2)), \
>                                       LocalActionDiagramIsUnimodular, \
>                                       true);
[ <LocalActionDiagram with 1 vertex and 1 arc>,
  <LocalActionDiagram with 1 vertex and 6 arcs>,
  <LocalActionDiagram with 1 vertex and 6 arcs>,
  <LocalActionDiagram with 1 vertex and 6 arcs>,
  <LocalActionDiagram with 1 vertex and 1 arc>,
  <LocalActionDiagram with 2 vertices and 2 arcs>,
  <LocalActionDiagram with 3 vertices and 4 arcs>,
  <LocalActionDiagram with 4 vertices and 6 arcs>,
  <LocalActionDiagram with 5 vertices and 8 arcs>,
  <LocalActionDiagram with 6 vertices and 10 arcs>,
  <LocalActionDiagram with 7 vertices and 12 arcs>,
  <LocalActionDiagram with 8 vertices and 14 arcs>,
  <LocalActionDiagram with 9 vertices and 16 arcs>,
  <LocalActionDiagram with 10 vertices and 18 arcs> ]
gap> Print(lad_list[3]);
Vertices = { 1 }
Arcs = {
        1 = ( origin = 1, terminus = 1, inverse = 1 )
        2 = ( origin = 1, terminus = 1, inverse = 2 )
        3 = ( origin = 1, terminus = 1, inverse = 3 )
```

```
        4 = ( origin = 1, terminus = 1, inverse = 4 )
        5 = ( origin = 1, terminus = 1, inverse = 6 )
        6 = ( origin = 1, terminus = 1, inverse = 5 )
}
Reverse Map = (5,6)
Vertex Labels = {
        1 = Group( [ (1,2) ] )
}
Arc Labels = {
        1 = [ 1, 2 ]
        2 = [ 3 ]
        3 = [ 4 ]
        4 = [ 5 ]
        5 = [ 6 ]
        6 = [ 7 ]
}
```

### 5.2.2 OneRSGraph/OneLocalActionDiagram

▷ `OneRSGraph(`*`[function_1, value_1, ...]`*`)` (operation)
▷ `OneLocalActionDiagram(`*`[function_1, value_1, ...]`*`)` (operation)

**Returns:** One RSGraphs or Local Action Diagrams satisfying the conditions given as arguments or `fail` if there are none satisfying them.

These functions are a shorthand for `AllRSGraphs([`*`function_1`*`, `*`function_2`*`, ...])[1]` and `AllLocalActionDiagrams([`*`function_1`*`, `*`function_2`*`, ...])[1]`. They are analogous to the `OneTransitiveGroup` function from the Transitive Groups package. Note that they do not offer any speed improvements to running the "`All`" variants of the function.

### 5.2.3 SetLocalActionDiagramDegreeTwoVertexBound

▷ `SetLocalActionDiagramDegreeTwoVertexBound(`*`bound`*`)` (operation)

Since any RSGraph and Local action diagram of degree two can be quickly manually constructed there needs to be a bound on the amount to use for the search functions (`AllRSGraphs` (5.2.1), `AllLocalActionDiagrams` (5.2.1)). By default this value is `10`. This can be changed using this function to *`bound`*.

### 5.2.4 LocalActionDiagramDebugSearch

▷ `LocalActionDiagramDebugSearch(`*`debug`*`)` (operation)

If *`debug`* is `true` then if there is an error in the search functions (`AllRSGraphs` (5.2.1), `AllLocalActionDiagrams` (5.2.1)) this will cause the error message to say which function and argument caused this error. It will not say what the error is.

# Chapter 6

# Extensions

We support converting between RSGraphs and digraphs from the Digraphs and graphs from the GRAPE. Converting from an RSGraph returns both the (di)graph and all data needed to convert back to the original RSGraph.

## 6.1 Digraphs Conversion

### 6.1.1 RSGraphToDigraph

▷ `RSGraphToDigraph(`*rsgraph*`)` (attribute)

**Returns:** A record containing the digraph and all data needed to reconstruct the RSGraph.

Given an RSGraph this function converts is to a digraph from the Digraphs package. A record is returned containing both the digraph and all data needed to reconstruct the original RSGraph. The record returned contains the elements:

- `digraph`: a digraph representing the graph structure (an object in the category `IsDigraph`.)
- `reverse_map`: the reverse map of the original RSGraph.
- `vertex_id_map`: a bijective map from the vertices of the original RSGraph to the vertices of the digraph which determines the vertex correspondence between the RSGraph and digraph.
- `arc_id_map`: a bijective map from the arcs of the original RSGraph to the edges of the digraph which determines the arc correspondence between the RSGraph and digraph.

As an example, if `2` is a vertex in the RSGraph then `2^vertex_id_map` is the corresponding vertex in the digraph. All digraphs returned by this function have vertices labelled from 1 to $n$ where $n$ is the number of vertices in the RSGraph. Arc `i` in the digraph is the arc in the `i`th position of `DigraphEdges(digraph)` --- i.e. `DigraphEdges(digraph)[i]`.

Example

```
gap> graph := RSGraphByAdjacencyList([[2, 3], [3, 2]], (1,2), [2, 3]);;
gap> digraph_rec := RSGraphToDigraph(graph);;
gap> digraph_rec.digraph;
<immutable digraph with 2 vertices, 2 edges>
gap> 2^digraph_rec.vertex_id_map;
1
```

### 6.1.2 RSGraphFromDigraph

▷ RSGraphFromDigraph(*digraph_rec*) (operation)

**Returns:** An RSGraph corresponding to the data in the input record.

Given the output from the function RSGraphToDigraph (6.1.1) (*digraph_rec*) this function returns the corresponding RSGraph. This returns an RSGraph that is exactly the same as the original RSGraph. If the data input to this function is not fro the RSGraphToDigraph function then there is no guarantee that this function will run correctly.

Example

```
gap> graph := RSGraphByAdjacencyList([[2, 3], [3, 2]], (1,2), [2, 3]);;
gap> digraph_rec := RSGraphToDigraph(graph);;
gap> new_graph := RSGraphFromDigraph(digraph_rec);;
gap> Print(graph);
Vertices = { 2, 3 }
Arcs = {
        1 = ( origin = 2, terminus = 3, inverse = 2 )
        2 = ( origin = 3, terminus = 2, inverse = 1 )
}
Reverse Map = (1,2)
gap> Print(new_graph);
Vertices = { 2, 3 }
Arcs = {
        1 = ( origin = 2, terminus = 3, inverse = 2 )
        2 = ( origin = 3, terminus = 2, inverse = 1 )
}
Reverse Map = (1,2)
```

### 6.1.3 RSGraphFromDigraph

▷ RSGraphFromDigraph(*digraph, reverse_map*) (operation)

**Returns:** An RSGraph.

Given a *digraph* and a *reverse_map* defined on the edges of the digraph this function returns an RSGraph with the same structure as the digraph and with the reverse map provided. This function works by using the adjacency listing of the digraph given by DigraphEdges(*digraph*). This means that the reverse map must be compatible with the order of the edges given by this adjacency listing.

All normal checks for an RSGraph construction are applied when using this function. In particular, this means that the digraph must be connected and must have a reverse arc for every arc in the digraph.

Example

```
gap> digraph := DigraphByEdges([[1, 1], [1, 2], [2, 1]]);;
gap> graph := RSGraphFromDigraph(digraph, (2, 3));;
gap> Print(graph);
Vertices = { 1, 2 }
Arcs = {
        1 = ( origin = 1, terminus = 1, inverse = 1 )
        2 = ( origin = 1, terminus = 2, inverse = 3 )
        3 = ( origin = 2, terminus = 1, inverse = 2 )
}
Reverse Map = (2,3)
```

## 6.2 GRAPE Conversion

For the sake of clarity, if an object in the following section is a graph from the GRAPE package then it is referred to as a GRAPE graph.

### 6.2.1 RSGraphToGRAPE

▷ RSGraphToGRAPE(*rsgraph*) (operation)

**Returns:** A record containing the GRAPE graph and all data needed to reconstruct the RSGraph.

Given an RSGraph this function converts is to a GRAPE graph. A record is returned containing both the GRAPE graph and all data needed to reconstruct the original RSGraph. The record returned contains the elements: - `graph`: a GRAPE graph representing the graph structure (an object for which `IsGraph` returns true.) - `reverse_map`: the reverse map of the original RSGraph. - `vertex_id_map`: a bijective map from the vertices of the original RSGraph to the vertices of the GRAPE graph which determines the vertex correspondence between the RSGraph and GRAPE graph. - `arc_id_map`: a bijective map from the arcs of the original RSGraph to the edges of the GRAPE graph which determines the arc correspondence between the RSGraph and GRAPE graph.

As an example, if 2 is a vertex in the RSGraph then `2^vertex_id_map` is the corresponding vertex in the GRAPE graph. All GRAPE graph returned by this function have vertices labelled from 1 to $n$ where $n$ is the number of vertices in the RSGraph. Arc `i` in the digraph is the arc in the `i`th position of `DirectedEdges(graph)` --- i.e. `DirectedEdges(graph)[i]`.

Example

```
gap> graph := RSGraphByAdjacencyList([[2, 3], [3, 2]], (1,2), [2, 3]);;
gap> graph_rec := RSGraphToGRAPE(graph);;
gap> graph_rec.graph;;
gap> 2^graph_rec.vertex_id_map;
1
```

### 6.2.2 RSGraphFromGRAPE

▷ RSGraphFromGRAPE(*graph_rec*) (operation)

**Returns:** An RSGraph corresponding to the data in *graph_rec*.

Given the output from the function `RSGraphToGRAPE` (6.2.1) (*graph_rec*) this function returns the corresponding RSGraph. This returns an RSGraph that is exactly the same as the original RSGraph. If the data input to this function is not fro the `RSGraphToGRAPE` function then there is no guarantee that this function will run correctly.

Example

```
gap> graph := RSGraphByAdjacencyList([[2, 3], [3, 2]], (1,2), [2, 3]);;
gap> graph_rec := RSGraphToGRAPE(graph);;
gap> new_graph := RSGraphFromGRAPE(graph_rec);;
gap> Print(graph);
Vertices = { 2, 3 }
Arcs = {
        1 = ( origin = 2, terminus = 3, inverse = 2 )
        2 = ( origin = 3, terminus = 2, inverse = 1 )
}
Reverse Map = (1,2)
gap> Print(new_graph);
Vertices = { 2, 3 }
```

```
Arcs = {
        1 = ( origin = 2, terminus = 3, inverse = 2 )
        2 = ( origin = 3, terminus = 2, inverse = 1 )
}
Reverse Map = (1,2)
```

### 6.2.3 RSGraphFromGRAPE

▷ RSGraphFromGRAPE(*graph*, *reverse_map*) (operation)

**Returns:** An RSGraph.

Given a GRAPE *graph* and a *reverse_map* defined on the edges of the GRAPE graph this function returns an RSGraph with the same structure as the GRAPE graph and with the reverse map provided. This function works by using the adjacency listing of the GRAPE graph given by DirectedEdges(*graph*). This means that the reverse map must be compatible with the order of the edges given by this adjacency listing.

All normal checks for an RSGraph construction are applied when using this function. In particular, this means that the GRAPE graph must be connected and must have a reverse arc for every arc in the GRAPE graph.

Example

```
gap> adj_mat := [[0, 1], [1, 0]];
gap> grape_graph := Graph(Group(()), [1, 2], OnPoints, \
>                         {x, y} -> adj_mat[x][y] = 1, true);;
gap> graph := RSGraphFromGRAPE(grape_graph, (1, 2));;
gap> Print(graph);
Vertices = { 1, 2 }
Arcs = {
        1 = ( origin = 1, terminus = 2, inverse = 2 )
        2 = ( origin = 2, terminus = 1, inverse = 1 )
}
Reverse Map = (1,2)
```

## 6.3 UGALY

The package UGALY implements Tornier's generalisation of the Burger-Mozes universal group to $k$-closures. In the case when $k = 1$ this corresponds to the original Burger-Mozes universal group. It represents these groups as objects in the category IsLocalAction.

As objects in the category IsLocalAction are also in the category IsPermGroup they can be directly used in LocalActionDiagramFromBurgerMozesUniversalGroup (2.1.4). We provide a function to convert a local action diagram representing a Burger-Mozes group to a local action object from UGALY.

### 6.3.1 LocalActionDiagramToUGALY

▷ LocalActionDiagramToUGALY(*lad*) (operation)

**Returns:** A local action.

Given a local action diagram *lad* corresponding to a Burger-Mozes group this returns the corresponding group as an object of the category IsLocalAction from the package UGALY. The local

action diagram *lad* must have a single vertex and have only self-reverse edges, and must also have at least one edge (see LocalActionDiagramIsBurgerMozesUniversalGroup (2.2.21).

Example

```
gap> lad := LocalActionDiagramFromBurgerMozesUniversalGroup(Group((1, 2), \
                                                              (3,4)));
<U(Group( [ (1,2), (3,4) ] )) (as a Local Action Diagram)>
gap> group := LocalActionDiagramToUGALY(lad);
Group([ (1,2), (3,4) ])
gap> LocalActionDegree(group);
4
gap> LocalActionRadius(group);
1
gap> LocalActionDiagramFromBurgerMozesUniversalGroup(group);
<U(Group( [ (1,2), (3,4) ] )) (as a Local Action Diagram)>
```

# Index